

📖 FedMedSeg: The Technical Mastery Guide

This document explains **everything** under the hood. It is written to turn you from a "newbie" into a "subject matter expert" overnight.

🔍 Module 1: The Model (Convolutional Neural Networks)

1.1 How does a computer "see" an X-ray?

To a computer, an image is just a grid of numbers. A 224x224 image is a matrix of 50,176 pixels.

- * ****0**** = Black

- * ****255**** = White

- * ****Pneumonia**** usually appears as "cloudy white patches" (consolidations) in the lungs.

1.2 What is a Convolution? (The "Feature Hunter")

A "kernel" (or filter) is a small 3x3 grid of numbers that slides over the image.

- * ****Mathematically:**** It performs a dot product (multiplication) at every step.

- * ****Visually:****

- * One filter might look for ****Horizontal Edges**** (ribs).

- * Another looks for ****Vertical Edges**** (spine).

- * Another looks for ****Curves**** (heart shape).

- * ****The Output:**** A "Feature Map" that highlights where these things are.

1.3 Why do we need "Depth"? (Layers)

- * ****Layer 1:**** Finds edges. (/ - |)

- * ****Layer 2:**** Combines edges to find ****Shapes****. (Circles, squares).

- * ****Layer 10:**** Combines shapes to find ****Structures****. (Lung lobes, heart).

- * ****Layer 50:**** Combines structures to find ****Pathologies****. (Infection patches).

- * ****Pooling (MaxPooling):**** This shrinks the image size (224 -> 112 -> 56). It keeps only the "strongest" features and ignores the noise. It says: "I don't care exactly *where* the pneumonia is, just that it *is* there."

🔄 Module 2: Transfer Learning (The Strategy)

2.1 MobileNetV2 Architecture

- * ****Why MobileNet?**** It uses a special trick called ****"Depthwise Separable Convolutions"****.

- * Normal Conv: Very expensive (Multiplies everything by everything).

- * Depthwise Conv: Splits the math into two cheaper steps.

- * ****Result:**** It runs \$10\times\$ faster than standard models but is almost as smart.

- * ****The "Bottleneck Residual Block":**** It squeezes the data small (bottleneck) to do math cheaply, then expands it back up. This is the secret sauce of efficiency.

2.2 Freezing Layers (The Theory)

When we say "Freeze Layer 1 to 100", we mean:

- * ****Code:**** ``param.requires_grad = False``

- * ****Math:**** During Backpropagation (learning), the Gradient (error signal) is blocked at this layer. The weights \$W\$ do not update.

- * ****Why?**** The first 100 layers (edges/shapes) are generic. A circle is a circle, whether in a cat photo or an X-ray. We do not need to waste computer power re-learning what a circle looks like.

🌐 Module 3: Federated Learning (The System)

3.1 The Fundamental Loop (FedAvg)

1. ****Selection:**** Server picks random clients (e.g., Hospital A & B).
2. ****Broadcast:**** Server sends the current Global Model (W_g) to clients.
3. ****Local Training:****
 - * Hospital A takes W_g and trains it on its own 100 X-rays.
 - * It calculates the "Loss" (error) and updates weights to W_a .
 - * $W_a = W_g - (\text{learning_rate} \times \text{gradient})$
4. ****Reporting:**** Clients send *only* the new weights (W_a and W_b) back to Server.
5. ****Aggregation:**** Server averages them:
$$W_{\text{new}} = \frac{n_A \cdot W_A + n_B \cdot W_B}{n_A + n_B}$$

(Where n is the number of samples—giving more weight to bigger hospitals).*

3.2 Non-IID Data (The Challenge)

- * ****IID:**** Independent and Identically Distributed. (A perfect world where every hospital is the same).
- * ****Non-IID:**** Real life. Pathological skew.
 - * ****Skew Scenario:**** Hospital A has 90% Pneumonia. Hospital B has 10% Pneumonia.
- * ****Problem:**** Client A's model will say "Predict Pneumonia for everything!". Client B's model will say "Predict Normal for everything!".
- * ****Aggregation Risk:**** Averaging two crazy extremes might produce a "dumb" middle ground.
- * ****Solution (FedProx):**** A better algorithm we will use later. It adds a "leash" (proximal term) to the math. It tells Client A: "You can learn from your data, but ****do not drift too far**** from the Global Model."

🛠️ Module 4: The Tech Stack (Why these tools?)

4.1 PyTorch

- * ****The Tensors:**** Everything in PyTorch is a Tensor (a multi-dimensional matrix) that lives on the GPU.
- * ****Autograd:**** PyTorch's magic engine. It records every math operation you do (addition, multiplication) in a graph. When you call `backward()`, it walks backward through this graph to calculate gradients automatically.

4.2 Flower (Flwr)

- * ****The Abstraction:**** Networking is hard. Sockets, gRPC, serialization, timeouts... Flower wraps all of this junk into a simple `start_client()` call.
- * ****Scalability:**** Flower is industrial-grade. It can handle 10 clients or 10 million mobile phones. Using it shows you are building "Production-Ready" software, not just a toy.

4.3 Opacus (Differential Privacy)

- * ****The Threat:**** "Model Inversion". If I have the gradients (weight updates), I can sometimes reconstruct the training image.
- * ****The Mechanism (DP-SGD):**** Differentially Private Stochastic Gradient Descent.
 1. ****Clipping:**** "Don't let any single image change the model too much." (Limits influence).
 2. ****Noise:**** "Add random static."
- * ****The Trade-off:**** More Privacy = Less Accuracy. (Because you are adding noise). Your job in Phase 3 is to measure this trade-off.

🛡 Module 5: Metrics (How we keep score)

5.1 F1-Score vs. Accuracy

- * **Why not just Accuracy?**
 - * Imagine a dataset with 99 healthy people and 1 sick person.
 - * A dumb model says "Everyone is Healthy!"
 - * Accuracy = 99%. **BUT** you missed the one person who needed help (The sick one).
- * **F1-Score:** It balances **Precision** (When I say sick, are they really sick?) and **Recall** (Did I find all the sick people?).
- * **In Medicine:** **Recall (Sensitivity)** is usually the most critical. You **never** want to miss a pneumonia case. It is better to have a False Alarm (False Positive) than a Missed Diagnosis (False Negative).

✅ Summary for Your Presentation

You built:

1. **A Brain:** MobileNetV2 (Partial Fine-Tuning) to efficiently see pneumonia.
2. **A System:** Federated Learning to allow collaboration without sharing data.
3. **A Simulation:** Partitioned Non-IID data to mimic real-world hospital challenges.
4. **A Baseline:** Proven that your efficient model (3B) matches heavy models (3C) in accuracy.